

30-Day Express + MongoDB Revision Challenge

Build One Project Every Day to Master Backend Development

Challenge Goal

Build 30 different projects from scratch, one per day, to reinforce Express.js and MongoDB concepts through hands-on practice.



Week 1: Foundation Projects (Days 1-7)

Day 1: Weather API Wrapper

- Build an API that fetches weather data from external API
- Store user's favorite cities in MongoDB
- GET/POST/DELETE favorite cities

Practice: Basic routing, external API calls, MongoDB CRUD

Day 2: URL Shortener

- Accept long URL, generate short code
- Redirect short code to original URL
- Track click counts

Practice: Route parameters, MongoDB queries, redirects

Day 3: Contact Form API

- Store contact submissions (name, email, message)
- GET all submissions, DELETE by ID
- Mark as read/unread

Practice: POST requests, validation, timestamps

Day 4: Simple Poll/Voting System

- Create polls with multiple options
- Vote on options, view results
- Prevent duplicate voting

Practice: Arrays in schemas, increment operations

Day 5: Quote of the Day API

- Store quotes with author and category
- Random quote endpoint
- Filter by category

Practice: Aggregation, random sampling

Day 6: Task Timer/Pomodoro Tracker

- Create tasks with time spent
- Start/stop timer sessions
- View total time per task

Practice: Date handling, calculations

Day 7: Bookmark Manager

- Save bookmarks (URL, title, tags)
- Search by tags
- Organize in folders

Practice: Arrays, text search, filtering



Week 2: User & Auth Projects (Days 8-14)

Day 8: Simple Note-Taking App

- User registration/login
- Each user has their own notes
- CRUD operations on notes

Practice: JWT auth, user-specific data

Day 9: Expense Tracker

- Users track expenses (amount, category, date)
- View total expenses, filter by date range
- Monthly/yearly summaries

Practice: Auth, date queries, aggregation

Day 10: Recipe Sharing API

- Users can post recipes
- Public recipes browsable by all
- Save favorite recipes

Practice: Public vs private data, population

Day 11: Habit Tracker

- Users create habits to track
- Mark habits as complete daily
- View streak count

Practice: Date logic, arrays, user relationships

Day 12: Simple Blog with Likes

- Users write posts
- Other users can like posts
- View most liked posts

Practice: References, counters, sorting

Day 13: Event RSVP System

- Create events
- Users RSVP (going/not going)
- View attendee list

Practice: Many-to-many relationships

Day 14: Password Manager API

- Store encrypted passwords for websites
- Users can only access their own passwords
- Search and organize passwords

Practice: Encryption, security, authorization



Week 3: Complex Features (Days 15-21)

Day 15: Forum/Discussion Board

- Topics with multiple replies
- Nested comments (replies to replies)
- Upvote/downvote system

Practice: Nested schemas, populations

Day 16: Job Board API

- Post jobs (company, position, salary)
- Users apply to jobs
- Track application status

Practice: Multiple relationships, status tracking

Day 17: Library Management System

- Books inventory
- Users borrow/return books
- Track due dates, overdue books

Practice: Status management, date comparisons

Day 18: Fitness Workout Logger

- Log workouts (exercise, sets, reps, weight)
- View progress over time
- Calculate personal records

Practice: Time series data, aggregations

Day 19: Movie Watchlist & Reviews

- Add movies to watchlist
- Rate and review watched movies
- View average ratings

Practice: Ratings, calculations, filtering

Day 20: Flash Card Learning App

- Create card decks
- Track which cards are mastered
- Spaced repetition logic

Practice: Complex schemas, algorithms

Day 21: Restaurant Review Platform

- Restaurants with multiple reviews
- Average rating calculation
- Filter by cuisine, rating

Practice: Aggregation pipelines, calculated fields



Week 4: Advanced Projects (Days 22-28)

Day 22: E-commerce Shopping Cart

- Products catalog
- Add/remove from cart
- Calculate total with discounts

Practice: Transactions, calculations

Day 23: Appointment Booking System

- Book time slots
- Prevent double booking
- Cancel appointments

Practice: Date/time logic, validations

Day 24: Social Media Feed

- Follow/unfollow users
- View feed of followed users' posts
- Like and comment

Practice: Complex queries, populations

Day 25: Quiz Application

- Create quizzes with questions
- Take quiz, get score
- Track user quiz history

Practice: Arrays, score calculation

Day 26: Real Estate Listings

- Post properties (price, location, features)
- Search by price range, location
- Save favorites

Practice: Range queries, geolocation basics

Day 27: Music Playlist Manager

- Create playlists
- Add songs to playlists
- Share playlists with other users

Practice: Many-to-many, sharing logic

Day 28: Inventory Management System

- Products with stock levels
- Orders that decrease stock
- Low stock alerts

Practice: Transactions, webhooks/notifications



Days 29-30: Full-Stack Challenges

Day 29: Chat Message Storage API

- Store chat messages between users
- Retrieve conversation history
- Mark messages as read
- Online/offline status

Practice: Real-time data structure, pagination

Day 30: Project Portfolio API

- Developers showcase projects
- Tags, technologies used
- Search and filter projects
- File uploads for screenshots

Practice: Everything combined!



Daily Structure for Maximum Learning

Morning (30 min)

- Plan the project structure
- Define routes and schema
- Write pseudocode

Afternoon (60-90 min)

- Code the complete project
- Test with Postman
- Handle errors

Evening (30 min)

- Add one bonus feature
- Refactor code
- Document in README
- Push to GitHub



Bonus Challenges (Add to Any Project)

1. Add pagination to any list endpoint
2. Add sorting (by date, name, rating, etc.)
3. Add search functionality
4. Add rate limiting to prevent abuse
5. Add email notifications
6. Add file uploads where relevant
7. Write 3-5 tests for main endpoints
8. Add API documentation with Swagger
9. Add caching with Redis for popular queries
10. Deploy to Render/Railway and get a live URL



Rules for Maximum Learning

- ✓ Start from scratch each day - no copying previous code
- ✓ No tutorials - use only documentation
- ✓ Time yourself - aim to complete in 2 hours
- ✓ Make it work first, then refactor
- ✓ Test every endpoint before moving on

- ✓ Git commit with message "Day X: [Project Name]"



Track Your Progress

- Create a GitHub repo: "30-days-express-mongodb"
- Each day is a separate folder
- Update main README with completed projects
- Note what you learned each day
- Share your progress on Twitter/LinkedIn



What You'll Master After 30 Days

- ✓ Express routing and middleware
- ✓ MongoDB schema design
- ✓ Authentication & authorization
- ✓ Data relationships (one-to-many, many-to-many)
- ✓ Query optimization
- ✓ Error handling
- ✓ File uploads
- ✓ Date/time handling
- ✓ Aggregation pipelines
- ✓ API best practices
- ✓ Testing basics
- ✓ Deployment



Difficulty Progression

- **Days 1-7:** Build confidence, basic CRUD
- **Days 8-14:** Add auth, user context
- **Days 15-21:** Complex relationships, business logic
- **Days 22-28:** Production-level features
- **Days 29-30:** Full capstone projects

Pro Tip: If you finish early any day, add a bonus feature or help someone in a Discord/Reddit community with their Express questions!

Good luck with your 30-day journey! 🚀